Contents

Take-Home Assignment #1 **2**

1. Computing and Visualizing Exact and Inversion-Computed Densities of the Student's t-Distribution 2
2. Computing and Visualizing the Sum of Two Student's t-Distributions via Convolution, Simulation, and Characteristic Function Inversion 4
3 and 4. Implementing and Computing Expected Shortfall (ES) for the Student's t-Distribution Using Analytical Formulas 6
5. Parametric and Nonparametric Bootstrap for Expected Shortfall Estimation for the Student's t-Distribution 8
6. Evaluating Coverage Probability and Interval Lengths for Parametric and Nonparametric Bootstrap Methods: A Simulation Study 11

Appendices **14**

A MATLAB Code for Task 1: t-Density Plot and Inversion Formula 14
B MATLAB Code for Task 2: Sum of Two t-Distributions 15
C MATLAB Code for Task 3 and 4: Expected Shortfall Calculation 17
D MATLAB Code for Task 5: Bootstrap Estimation 18
E MATLAB Code for Task 6: Coverage Probability Simulation 20

Take-Home Assignment #1

All solutions were implemented using MATLAB. The reader can find the full comprehensive code attached in the appendix. For structural simplicity, the code provided in the report body does not contain any comments or plot functions sometimes leaving out certain parts of the code. It is solely for the reader to follow the intuition of the results provided while reading.

1. Computing and Visualizing Exact and Inversion-Computed Densities of the Student's t-Distribution

The primary objective of this task was to develop a MATLAB-based computational tool that accepts a user-defined degree of freedom value, validates its positivity, and subsequently plots both the exact p.d.f. and an approximation derived using the inversion formula. This visualization aims to facilitate a comparative analysis between the exact and inversion-computed densities across a range of x-values that sufficiently capture the distribution's significant features. Specifically, the x-axis range adapts based on the degree of freedom to encompass the majority of the density, extending from -6 to 6 for low degrees of freedom (e.g., df=1, analogous to the Cauchy distribution) and narrowing to -3 to 3 for higher degrees of freedom (e.g., df=30), where the distribution increasingly approximates the normal distribution. The inversion formula for recovering a probability density function (p.d.f.) from its characteristic function (c.f.) is given by:

$$f_X(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} e^{-itx} \varphi_X(t)\, dt \tag{1}$$

For symmetric distributions, such as the Student's t-distribution, the inversion formula can be simplified. Since the Student's t-distribution is symmetric and its density is real-valued, we can simplify this to:

$$f_X(x) = \frac{1}{\pi} \int_{0}^{\infty} \cos(tx) \varphi_X(t)\, dt \tag{2}$$

We will make use of this result as demonstrated in Listing 1, where the inversion formula is calculated using Equation (2). The characteristic function $\varphi_X(t)$ called in line 3 of Listing 1 makes use of the Bessel function $K_\nu(z)$ of second kind and the Gamma function.

$$\varphi_X(t) = \begin{cases} 1, & \text{if } t = 0, \\ \frac{z^\nu K_\nu(z)}{\Gamma(\nu) \cdot 2^{\nu-1}}, & \text{if } t \neq 0, \end{cases} \tag{3}$$

where:

$$z = \sqrt{\mathrm{df}} \cdot |t|, \quad \nu = \frac{\mathrm{df}}{2}$$

and the Bessel function $K_\nu(z)$ as derived in *Linear Models and Time-Series Analysis* (M. Paolella, 2019). I have used the Bessel function of second kind that is built into MATLAB. $\varphi_X(t)$ is real because X is symmetric about zero.

Listing 1: MATLAB Code for the Inversion Formula

```
1 function density = inversion_formula(x,df)
2     t_max = 100;
3     integrand = @(t) cos(t*x).* phi_function(t,df);
4     density = (1/pi)*integral(integrand,0,t_max,'ArrayValued',true);
5 end
```

As shown in Listing 3, I handle the case of $t = 0$ separately to avoid NaNs since I ran into some errors when not doing so. To calculate the density using our inversion formula as stated in Listing 1 we are running a loop as shown in Listing 2, i.e. integrate numerically here. The code in the appendix shows how Figure 1 was created using conditional ranges for x-values. When df is small, the Student's t-distribution has heavier tails. This means that the distribution has more probability mass far away from the mean (closer to the tails). The tails of the distribution extend further out, so to capture the behavior of the distribution accurately, the integration is performed over a wider range (from -6 to 6 in this case). A wider range is necessary because the function does not decay quickly, and the density remains significant even at values of x that are far from the mean. As df increases, the Student's t-distribution approaches a normal (Gaussian) distribution, which has much thinner tails. When df is large, the distribution is concentrated more closely around the mean, and the tails decay rapidly. In this case, integrating over a narrower range (from -3 to 3) is sufficient because the density becomes negligible outside this range. This makes the computation more efficient while still capturing the essential characteristics of the distribution.

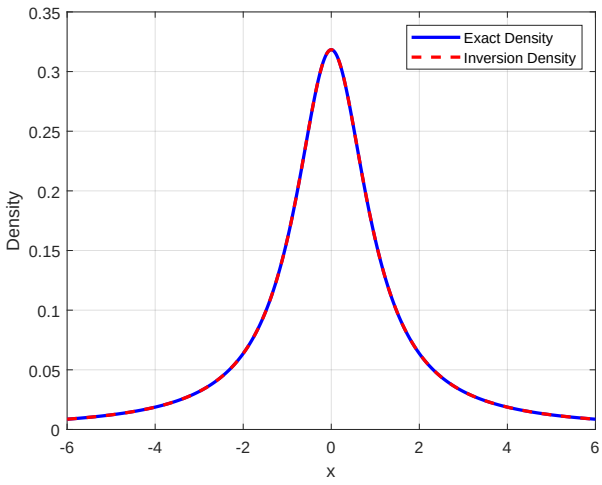
Listing 2: MATLAB Code for the Exact Density and Inversion Density

```
exact_density = tpdf(x, df);
inversion_density = zeros(size(x));
for i = 1:length(x)
    inversion_density(i) = inversion_formula(x(i),df);
end
```

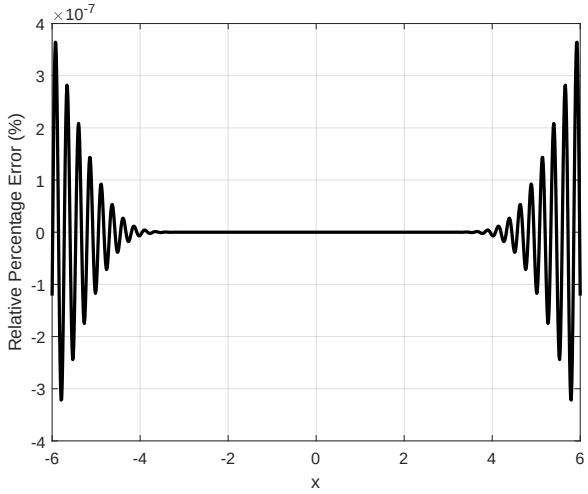
Listing 3: MATLAB Code for Characteristic Function

```
function phi = phi_function(t, df)
    nu = df/2;
    z = sqrt(df)*t;
    phi = zeros(size(t));
    zero_idx = (t == 0);
    phi(zero_idx) = 1;
    pos_idx = ~zero_idx;
    z_pos = z(pos_idx);
    K_nu = besselk(nu, z_pos);
    num = (z_pos).^nu .* K_nu;
    denom = gamma(nu)*2^(nu-1);
    phi(pos_idx) = num/denom;
end
```

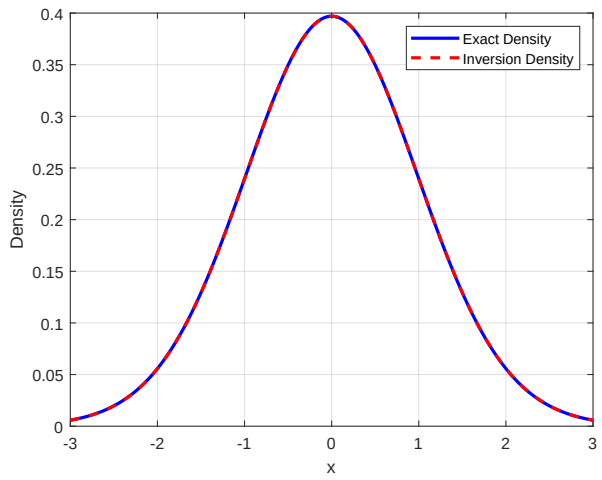
Figure 1 shows the exact and inversion-computed densities as well as the relative percentage error. As can be seen in 1a, using one degree of freedom for our student's t-distribution, the density calculated using the exact approach and the one based on the inversion formula closely align. Figure 1b shows the relative percentage error between the density computed by using the inversion formula and the exact density and is defined as $100 \cdot$(Approx - True)/True. We can see oscillating behavior in the tails although in the area of $\times 10^{-}7$. Figures 1c and 1d show the result when choosing *df=50*. This completes the first part of the assignment.



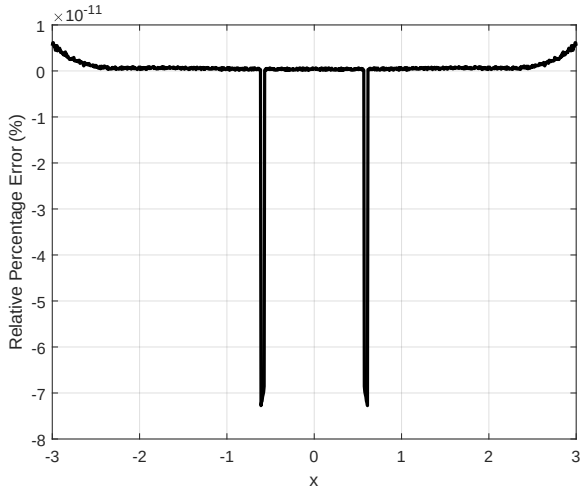
(a) Densities for $df = 1$



(b) Relative Percentage Error



(c) Densities for $df = 50$



(d) Relative Percentage Error

Figure 1: Comparison of Exact and Inversion-Computed Densities of the Student's t-Distribution and the Relative Percentage Error

2. Computing and Visualizing the Sum of Two Student's t-Distributions via Convolution, Simulation, and Characteristic Function Inversion

The goal of this part is to compute and visualize the sum of two independent Student's t-distributions using multiple methods to achieve this: convolution, simulation, and characteristic function inversion.

First, we simulate the sum of two independent Student's t-Distributions by simulating two independent student's t-distributions and compute the sum of these sample distributions as shown in Listing 4. We can use the *ksdensity* function in MATLAB to later combine the three approaches overlaying each line based on the different approaches. Also, we use N, i.e. the sample size as an input for our simulated sum function in order to see how the simulation approximates to the convolution and CF inversion density closer as N increases, shown in Figure 2.

Listing 4: MATLAB Simulation of Sum of two independent Student's t-distributions

```
function density = simulate_sum_density(z, df1, df2, N)
    X = trnd(df1, N, 1);
    Y = trnd(df2, N, 1);
    Z = X+Y;
    [density_values, density_points] = ksdensity(Z, z, 'Function', 'pdf', '
        Bandwidth', 0.1);
    density = interp1(density_points, density_values, z, 'linear', 0);
end
```

In Listing 5 we are turning to the characteristic function inversion formula. Again, we make use of the function provided in Listing 3 to calculate the combined characteristic function. Since we know that for independenten random variables

$$\varphi_{X+Y}(t) = \varphi_X(t) \cdot \varphi_Y(t)$$

where $\varphi_X(t)$ is the characteristic function of random variable X, the inversion formula can be applied. We define the integrand as

$$\varphi_{X+Y}(t) \cdot e^{-i \cdot t \cdot x}$$

and compute the integral for this integrand. After doing some research I found that using the *trapz* function numerically integrates the product of the characteristic function and the complex exponential over t. This approximates the integral in the characteristic function inversion formula and yields the best results for my simulation.

Listing 5: MATLAB Code of Characteristic Function Inversion Formula

```
function density = cf_inversion_density(z, df1, df2)
    t_max = 100;
    N_t = 10000;
    t = linspace(-t_max, t_max, N_t);
    phi_combined = phi_function(t, df1).* phi_function(t, df2);
    density = zeros(size(z));
    for i = 1:length(z)
        x = z(i);
        integrand = phi_combined.* exp(-1i*t*x);
        density(i) = (1/(2*pi))*trapz(t, integrand);
    end
    density = real(density);
end
```

In Listing 6 we are computing the integral convolution formula for sums of two independent Student t random variables using numerical integration function *quadgk* in MATLAB. In the case of two t-distributions, we calculate the convolution of their probability density functions (PDFs) to obtain the resulting PDF of their sum. I use the @ symbol in MATLAB to reference a function without actually executing it which allows us to, i.e., pass functions as arguments to other functions or store functions in variables. I will make use of this throughout most of our MATLAB codes which is mainly useful for the full codes provided in the Appendix.

Listing 6: MATLAB Code for Integral Convolution Formula

```
function density = convolution_density(z,df1,df2)
    integrand = @(x) tpdf(x,df1).* tpdf(z-x,df2);
    density = quadgk(integrand, -Inf, Inf, 'RelTol', 1e-6, 'AbsTol', 1e-9);
end
```

We define the integrand in Listing 6 as we have seen in the first part of our lectures as

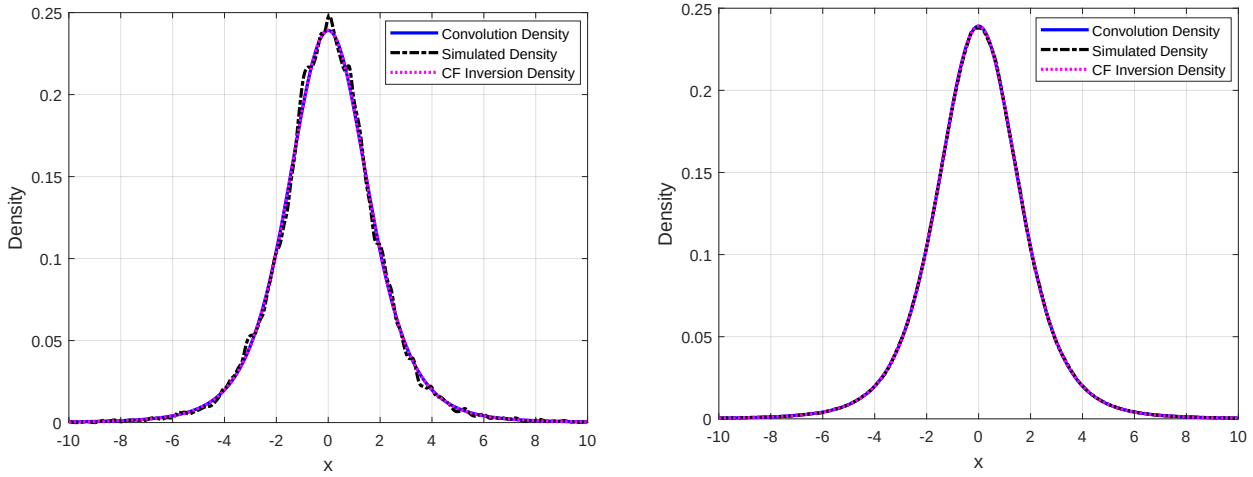
$$f_X(x) \cdot f_Y(z-x)$$

where $Z = X + Y$ and $f_X(x), f_Y(y)$ are the pdfs of the two student's t-distributions. Then by convolution (derived using double integrals or a jacobian transformation) we know that

$$f_Z(z) = \int_{-\infty}^{\infty} f_X(x) \cdot f_Y(z-x)dx$$

by independence of X and Y. The computation of the integral is done by using *quadgk* in MATLAB. I found that MATLAB's *quadgk* for adaptive quadrature over the real line actually makes sure that the computation works despite the t-distribution's heavy tails (see MATLAB function description). So we don't have to worry about that.

Figure 2 completes this second part of the assignment showing the Convolution Density, Simulated Density and Characteristic Function Inversion Formula Density for two different simulation sizes. We can see that for large N, as expected, the simulated density (as obtained from the ksdensity function) approximates to the other two methods stated above and are essentially overlapping which completes this part of the assignment.



(a) Simulation Size $N = 10000$

(b) Simulation Size $N = 1000000$

Figure 2: Comparison for two Different Simulation Sizes

3 and 4. Implementing and Computing Expected Shortfall (ES) for the Student's t-Distribution Using Analytical Formulas

In this part of the assignment I will combine Exercise 3 and 4 as they are related. Also, I will refer and use codes provided in this section for Exercise 5 and 6 as Exercises 3 and 4 are mainly pre-requisites for what follows. As stated in Fundamental Statistical Inference (M. Paolella, 2018) Value at Risk (VaR) and Expected Shortfall (ES) are widely used tail risk measures. ES is of particular importance due to its classification as a coherent risk measure, which satisfies the subadditivity property.

Using the true ES formula as stated in Appendix 8 of Fundamental Statistical Inference (M. Paolella, 2018) gives us the formula to compute the true Expected Shortfall for a Student's t-distribution using Listing 7.

Listing 7: MATLAB Code for Computation of True Expected Shortfall

```
VaR = tinv(alpha, nu);
pdf_VaR = tpdf(VaR, nu);
ES = -((nu+VaR^2)/((nu-1)*alpha))*pdf_VaR;
```

To program a function to input the parameters of the Student t (location, scale, df, and ESlevel), where ESlevel is the number in (0,1) indicating the tail probability returning the true Expected Shortfall Value we use Listing 8. First, we define the tail probability / the ESlevel as (commonly used) α. Since the ES is important to determine the potential "Loss" I multiply it by *-1*. Then we compute the PDF at the Value at Risk (VaR) for the t-distribution and henceforth can compute the Expected Shortfall for the t-distribution using the correct formula as stated in Listing 7. Here I make use of the fact that if Z is a location-zero, scale-one random variable and $Y = \sigma \cdot Z + \mu$ for $\sigma > 0$, then

$$ES(Y) = \mu + \sigma \cdot ES(Z)$$

and obviously (given a VaR)

$$ES(Z) = E[Z|Z < VaR]$$

Figure 3 shows a basic visualization of the Expected Shortfall and Value at Risk for the student's t-distribution based on 5 degrees of freedom and a 95 percent confidence level. Since the degrees of freedom determine tail behavior of the student's t-distribution, we expect that a change in the degrees of freedom will have an impact on the resulting sample distribution of the Expected Shortfall that we will simulate in Exercise 5. Lastly we adjust for location and scale parameters for the Expected Shortfall. Again, the full code is provided in the appendix where standard values for location and scale are set to 0 and 1 respectively. This completes the third and fourth part of the assignment.

Listing 8: MATLAB Code for Function to the Parameters of the Student's t

```
function ES = compute_ES_student_t(nu, location, scale, ESlevel)
    alpha = ESlevel;
    VaR_standard = tinv(alpha, nu);
    pdf_VaR_standard = tpdf(VaR_standard, nu);
    ES_standard = -((nu+VaR_standard^2)/((nu-1)*alpha))*pdf_VaR_standard;
    ES = location+scale*ES_standard;
end
```

As mentioned before, Figure 3 shows the VaR and ES for the 95 percent confidence level and 4 degrees of freedom. The true ES is calculated using the code provided in 8 that takes the VaR (which is necessary to calculate the ES) and the degrees of freedom as an input. We can see that the ES is basically the "expectation in the tail" of the distribution based on the 95 percent confidence level. The VaR and ES for the given degrees of freedom and confidence level is provided in the graphc as -2.13 and -3.20 respectively. So given we have "exceeded" the VaR the expected "loss" in the context of finance is -3.20. The full code for Exercise 4 can be found in the appendix where I define a function that takes degrees of freedom, location, scale and ESLevel as an input and then computes the respective *true* expected shortfall based on Listing 8.

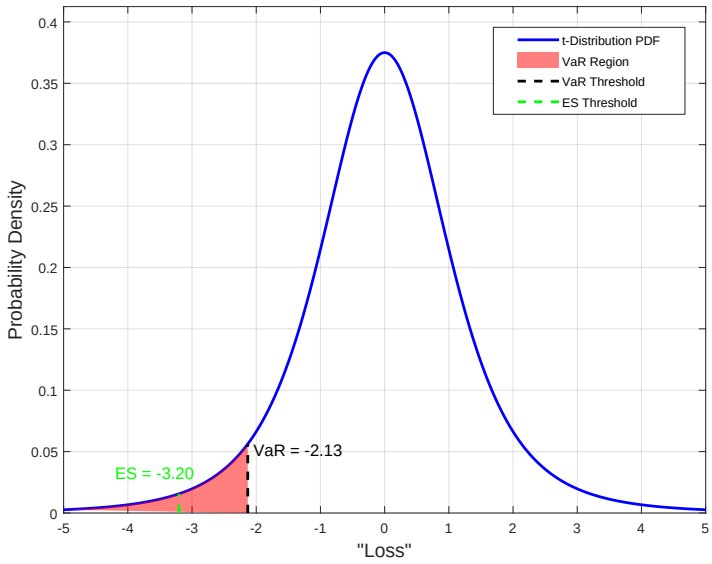


Figure 3: True Expected Shortfall for df=4 and 95 Percent Confidence Level

Figure 3 was created using Listing 18 in the appendix. The code takes the degrees of freedom nu and the confidence level $1 - \alpha$ as an input and returns the true VaR and ES.

5. Parametric and Nonparametric Bootstrap for Expected Shortfall Estimation for the Student's t-Distribution

In this part of the assignment we are using boostrapping as an estimation method of the expected shortfall for the student's t-distribution. First, we generate a random data set of IID Student's t realizations (with location zero and scale 1 and the passed df value) using the trnd() function as shown in Listing 9 while setting the location parameter to 0 and scale parameter to 1.

Listing 9: Generate a Random Data Set of IID Student's t Realizations in MATLAB

```
data = trnd(df, n, 1);
```

Using the random data set of IID Student's t realizations with location zero and scale parameter 1 and pre-defined degrees of freedom we continue generating a 90 percent confidence interval of the ES, based on the parametric bootstrap. First, we make use of the maximum likelihood estimator (MLE) applying it to the simulated data set which results in an estimate for each parameter, namely the degrees of freedom, the location and the scale as shown in Listing 10. The function inputs the data defined before and an amount of resamples performed during the bootstrap defined as B. In the parametric case, this is not really a "resample" but rather a new simulation. Figure 4 shows a histogram of the scale and degrees of freedom maximum likelihood estimators for N = 250 and B = 500 simulations resulting from running Listing 10 showing asymptotic behavior regarding the true values of location and scale. The same result applies to the degrees of freedom. Since we compute bootstrap_sample_p based on the sample we include location and scale (that we obtain from the MLE) in the bootstrap resamlpe.

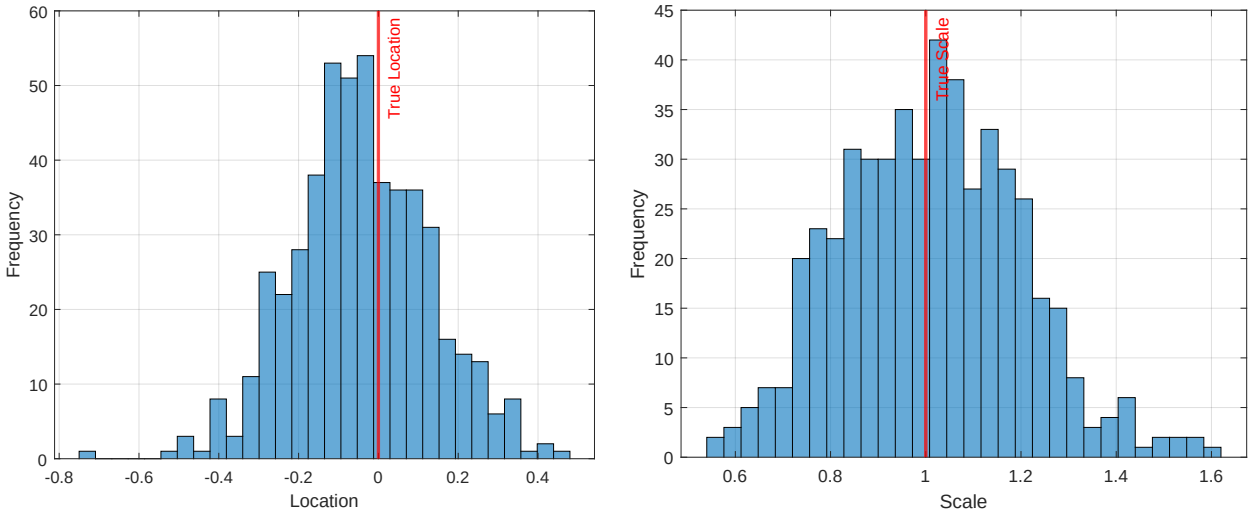
The parametric bootstrap presented in Listing 10 first estimates location, scale and degrees of freedom, then uses the estimates as inputs in the bootstrap to simulate a new sample based on the maximum likelihood estimator, i.e. based on the initial data as obtained from Listing 9. We then simulate B replications of this process and save the resulting expected shortfall for each b in 1:B.

Based on the resulting ES sample distribution, we can calculate empirical quantiles by taking the 5th and 95th percentile, i.e. cutting off the top 5 and bottom 5 percent of the sample distribution which we define as our parametric confidence interval.

Listing 10: MATLAB Code for Parametric Bootstrap

```
function CI_parametric = compute_ES_parametric_bootstrap_single(data, B)
    ESlevel = 0.05;
    mle_params = mle(data, 'distribution', 'tLocationScale');
    nu_mle = mle_params(3);
    location_mle = mle_params(1);
    scale_mle = mle_params(2);
    ES_bootstrap_p = zeros(B, 1);
    for b = 1:B
        bootstrap_sample_p = location_mle+scale_mle*trnd(nu_mle, length(data), 1);
        VaR_bootstrap_p = quantile(bootstrap_sample_p, ESlevel);
        ES_bootstrap_p(b) = mean(bootstrap_sample_p(bootstrap_sample_p <=
            VaR_bootstrap_p));
    end
    lower_p = prctile(ES_bootstrap_p, 5);
    upper_p = prctile(ES_bootstrap_p, 95);
    CI_parametric = [lower_p, upper_p];
end
```

This simulation is performed once in this exercise, then repeated *sim* times in Exercise 6 in order to obtain a sample distribution of the parametric confidence intervals.



(a) Sample distribution of location

(b) Sample distribution of scale

Figure 4: MLE Sample Distribution for *N=250, B=500*

I also tried using the *random* function from MATLAB by specifying the student's t-distribution using the MLE results shown in Listing 11 to compute the parametric bootstrap replication which after all yields the same result for the parametric bootstrap.

Listing 11: MATLAB Code for Nonparametric Bootstrap

```
bootstrap_sample_p = random('tLocationScale', location_mle, scale_mle, nu_mle,
    length(data), 1);
```

Repeating this exercise now with the the nonparametric bootstrap and using the empirical estimator of the ES we obtain Listing 12. For each b in 1:B we resample from our intial data as obtained from Listing 9 and then calculate the Expected Shortall based on the resulting sample distribution from resampling. Using this sample distribution of the ES, we again calculate a confidence interval using the same approach as mentioned before.

Listing 12: MATLAB Code for Nonparametric Bootstrap

```
function CI_nonparametric = compute_ES_nonparametric_bootstrap_single(data, B, n)
    ES_bootstrap_np = zeros(B, 1);
    ESlevel = 0.05;
    for b = 1:B
        ind = unidrnd(n, [n, 1]);
        bootsamp = data(ind);
        VaR_bootstrap_np = quantile(bootsamp, ESlevel);
        ES_bootstrap_np(b) = mean(bootsamp(bootsamp <= VaR_bootstrap_np));
    end
    lower_np = prctile(ES_bootstrap_np, 5);
    upper_np = prctile(ES_bootstrap_np, 95);
    CI_nonparametric = [lower_np, upper_np];
end
```

Figure 5 shows a Boxplot of the resulting Expected Shortfall sample distribution. It is crucial to note that the boxplot is based on **one** sample distribution. Here, the parametric bootstrap clearly performs better than the nonparametric bootstrap as it is centered more close to the true value of the expected shortfall as obtained from running Listing 7 while the nonparametric estimation returns a more spread out while the parametric is closer in mean and spread of the sample ES distribution. In Exercise 6 we will repeat this simulation *1000* times and find that the true coverage based on confidence intervals is better for the parametric bootstrap but the average length of the confidence intervals are smaller for the nonparametric bootstrap contrary to what we see in this picture.

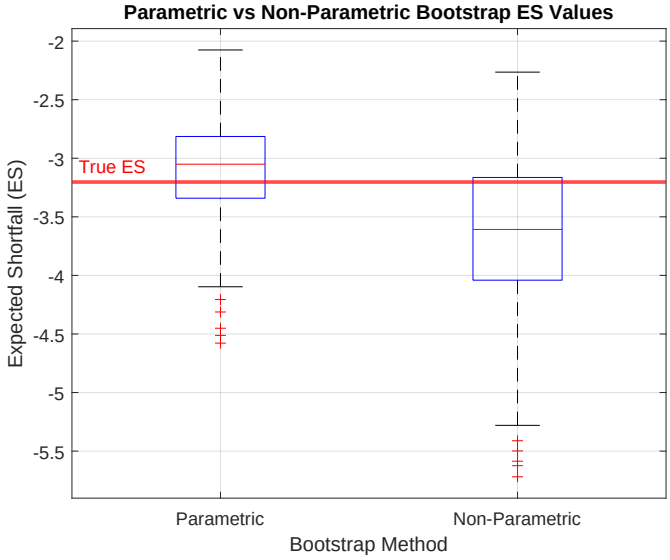


Figure 5: Boxplot for ES Sample Distribution based on Parametric and Nonparametric Bootstrap

We complete this exercise by including Table 1 showing confidence intervals based on 5 repititons of Listings 10 and 12. The first two columns are lower and upper confidence interval bounds for the

parametric bootstrap. The nonparametric CI bounds are shown in columns 3 and 4. We see that generally, the intervals appear greater for the parametric bootstrap in this sample which we will verify in Exercise 6.

Parametric Bootstrap		Nonparametric Bootstrap	
Lower Bound	Upper Bound	Lower Bound	Upper Bound
-3.4862	-2.3281	-3.0513	-2.4556
-4.2448	-2.5051	-4.1135	-2.4358
-3.2733	-2.0935	-3.4060	-1.9540
-4.5001	-2.4745	-4.2867	-2.9300
-4.5770	-2.4932	-4.1737	-2.7254

Table 1: Some Confidence Intervals from the Parametric and Nonparametric Bootstrap

The MLE for *tLocationScale* in MATLAB delivers the location first, then the scale and thirdly the shape as can be seen in the MathWorks documentation of the mle() function.

6. Evaluating Coverage Probability and Interval Lengths for Parametric and Nonparametric Bootstrap Methods: A Simulation Study

In this exercise we will repeat Listings 10 and 12 *sim=1000* times, store each resulting confidence interval in a variable and report the average confidence interval length as well as the empirical coverage probability for both the parametric and nonparametric bootstrap. In Listing 12 we simulate the previous exercise *1000* times. For each repetition, we compute the length of the parametric and nonparametric bootstrap confidence interval, hence obtain a distribution of 1000 different lengths for each. Apart from that, we also compute coverage probabilities based on the resulting confidence interval in each repetition by checking whether the True ES as obtained from Listing 7 is within the empirical confidence interval. If so, MATLAB stores a value of 1 in that case and 0 otherwise.

Listing 13: Simulation of 1000 Confidence Intervals

```
for s = 1:sim
    CI_parametric = compute_ES_parametric_bootstrap_single(data, B);
    coverage_parametric(s) = (CI_parametric(1) < ES_true) && (CI_parametric(2) >
        ES_true);
    length_parametric(s) = CI_parametric(2)-CI_parametric(1);
    CI_nonparametric = compute_ES_nonparametric_bootstrap_single(data, B, n);
    coverage_nonparametric(s) = (CI_nonparametric(1) < ES_true) && (
        CI_nonparametric(2) > ES_true);
    length_nonparametric(s) = CI_nonparametric(2)-CI_nonparametric(1);
end
```

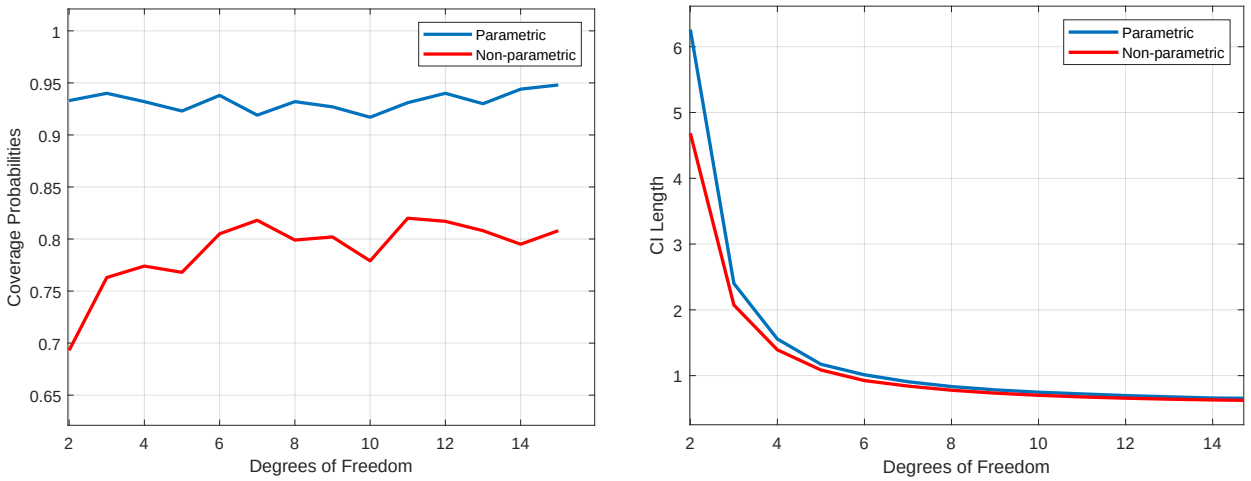
We then continue with Listing 14 where we take the average of the coverage vector and the length vector that result from running Listing 13.

Listing 14: Average Actual Coverage and average CI Length

```
empirical_coverage_parametric = mean(coverage_parametric)*100;
empirical_coverage_nonparametric = mean(coverage_nonparametric)*100;
avg_length_parametric = mean(length_parametric);
avg_length_nonparametric = mean(length_nonparametric);
```

In Figure 6 we repeat Listing 12 and Listing 13 for different degrees of freedom imputed in the student's t-distribution that we obtain our sample data from. We can see that with increasing

degrees of freedom due to different tail behavior of the student's t the average CI length as reported in 6b decreases. It is also noteworthy that the average CI length is bigger for the parametric than the nonparametric. Also, the parametric bootstrap coverage probabilities perform much better than the nonparametric ones as can be seen in 6a. The nonparametric bootstrap coverage probability increases with the number of degrees of freedom which completes Exercise 6.



(a) Coverage Probability

(b) Average CI length

Figure 6: Coverage Probabilities and average CI length for different degrees of freedom given *N=250, B=500*

The results for the parametric and nonparametric bootstrap given *4 degrees of freedom*, *N=250, B=500* based on *1000 simulations* are provided in Table 2. We see that, as shown in Figure 6a, the parametric bootstrap has a greater coverage probability.

Method	Average CI Length	Coverage Probability (%)
Parametric	1.5338	93.20
Nonparametric	1.3561	77.50

Table 2: Comparison of Parametric and Nonparametric Bootstrap Methods

Using *N=10000, B=500, df=4, sim=1000* I obtain the results presented in Table 3 (which took quite a while to simulate). So clearly, as can be expected, we obtain much "smaller" average CI length and improved coverage probability for the nonparametric bootstrap. The parametric bootstrap is consistently above the nominal 90 percent coverage.

Method	Average CI Length	Coverage Probability (%)
Parametric	0.2559	94.00
Nonparametric	0.2544	89.00

Table 3: Comparison of Parametric and Nonparametric Bootstrap Methods

Performing an "m out of n" nonparametric bootstrap as proposed by Peter J. Bickel and Anat Sakov (2008) can change the outcome of the results in Ex.6 as shown in Figure 7 where we show how the average CI lenght for the nonparametric bootstrap changes depending on the selection of m in the

"m out of n" nonparametric bootstrap. We see that, for example, choosing *m=150* puts it closer to the average CI length of the parametric bootstrap provided in Figure 6b. Peter J. Bickel and Anat Sakov (2008) suggest further restrictions on m and n, i.e. that $m/n \to 0$ as $m \to \infty$. Since this is not part of Exercise 6, I will not continue further on this. So to summarize, sample size, distribution of the data, estimation methods, bootstrap variability all have an impact on the empirical coverage probability and average CI length.

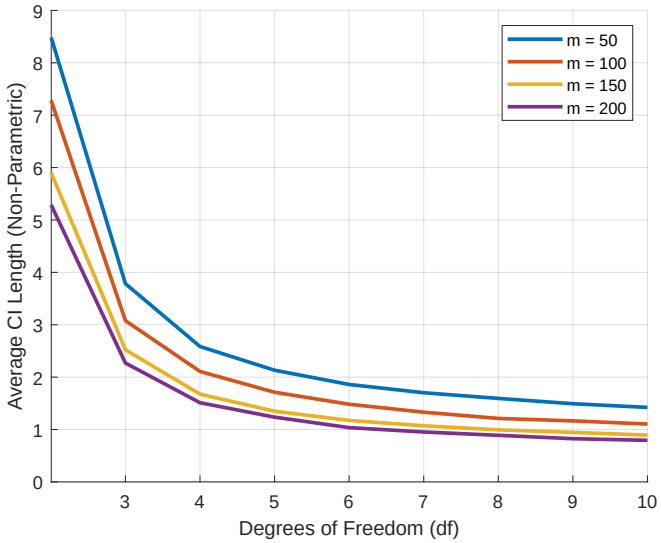


Figure 7: "m out of n" Simulation for Nonparametric Bootstrap

The full code for the main part of Exercise 6 is shown in Listing 20. This completes the report. As a last note, I attached Listing 21 to compute the results shown in Figure 7 which is just the altered code from previously now using just m out of n bootstrap resamples.

Appendices

MATLAB Code for Task 1: t-Density Plot and Inversion Formula

Listing 15: MATLAB Code for t-Density Plot

```
function plot_t_density()
    df = input('Enter df: ');
    if df < 30
        x = linspace(-6, 6, 1000);
    else
        x = linspace(-3, 3, 1000);
    end
    exact_density = tpdf(x, df);
    inversion_density = zeros(size(x));
    for i = 1:length(x)
        inversion_density(i) = inversion_formula(x(i), df);
    end
    figure;
    plot(x, exact_density, 'b', 'LineWidth', 5.0); hold on;
    plot(x, inversion_density, 'r', 'LineWidth', 2.0);
    xlabel('x');
    ylabel('Density');
    title(['Density Comparison for df = ' num2str(df)]);
    legend('Exact Density', 'Inversion Density');
    grid on;
    relative_error = (inversion_density-exact_density) ./ exact_density*100;
    figure;
    plot(x, relative_error, 'k', 'LineWidth', 2.0);
    xlabel('x');
    ylabel('Relative Percentage Error (%)');
    title('Relative Percentage Error between Densities');
    grid on;
end
function density = inversion_formula(x, df)
    t_max = 100;
    integrand = @(t) cos(t*x) .* phi_function(t, df);
    density = (1/pi) * integral(integrand, 0, t_max, 'ArrayValued', true);
end
function phi = phi_function(t, df)
    nu = df/2;
    z = sqrt(df)*t;
    phi = zeros(size(t));
    zero_idx = (t == 0);
    phi(zero_idx) = 1;
    pos_idx = ~zero_idx;
    z_pos = z(pos_idx);
    K_nu = besselk(nu, z_pos);
    num = (z_pos).^nu .* K_nu;
    denom = gamma(nu)*2^(nu-1);
    phi(pos_idx) = num/denom;
end
```

MATLAB Code for Task 2: Sum of Two t-Distributions

Listing 16: MATLAB Code for Expected Shortfall

```
function plot_sum_t_distributions_two_simulations()
    df1 = input('Enter df1: ');
    df2 = input('Enter df2: ');
    if df1 < 30 || df2 < 30
        z = linspace(-10, 10, 1000);
    else
        z = linspace(-6, 6, 1000);
    end
    conv_density = zeros(size(z));
    for i = 1:length(z)
        conv_density(i) = convolution_density(z(i), df1, df2);
    end
    [normal_density, total_var] = normal_approx_density(z, df1, df2);
    cf_density = cf_inversion_density(z, df1, df2);
    N_large = 1e6;
    N_small = 1e4;
    sim_density_large = simulate_sum_density(z, df1, df2, N_large);
    sim_density_small = simulate_sum_density(z, df1, df2, N_small);
    figure;
    plot(z, conv_density, 'b', 'LineWidth', 2.0); hold on;
    plot(z, sim_density_large, 'k-.', 'LineWidth', 2.0);
    plot(z, cf_density, 'm:', 'LineWidth', 2.0);
    xlabel('z', 'FontSize', 12);
    ylabel('Density', 'FontSize', 12);
    legend('Convolution Density', 'Simulated Density', 'CF Inversion Density', '
        Location', 'Best');
    grid on;
    figure;
    plot(z, conv_density, 'b', 'LineWidth', 2.0); hold on;
    plot(z, sim_density_small, 'k-.', 'LineWidth', 2.0);
    plot(z, cf_density, 'm:', 'LineWidth', 2.0);
    xlabel('z', 'FontSize', 12);
    ylabel('Density', 'FontSize', 12);
    legend('Convolution Density', 'Simulated Density', 'CF Inversion Density', '
        Location', 'Best');
    grid on;
    set(gcf, 'Position', [100, 100, 1200, 500]);
end
function density = convolution_density(z, df1, df2)
    integrand = @(x)tpdf(x, df1) .*tpdf(z-x, df2);
    density = quadgk(integrand, -Inf, Inf, 'RelTol', 1e-6, 'AbsTol', 1e-9);
end
function density = simulate_sum_density(z, df1, df2, N)
    X = trnd(df1, N, 1);
    Y = trnd(df2, N, 1);
    Z = X+Y;
    [density_values, density_points] = ksdensity(Z, z, 'Function', 'pdf', '
        Bandwidth', 0.1);
    density = interp1(density_points, density_values, z, 'linear', 0);
end
function density = cf_inversion_density(z, df1, df2)
    t_max = 100;
    N_t = 10000;
    t = linspace(-t_max, t_max, N_t);
    phi_combined = phi_function(t, df1) .*phi_function(t, df2);
```

```
    density = zeros(size(z));
    for i = 1:length(z)
        x = z(i);
        integrand = phi_combined .* exp(-1i*t*x);
        density(i) = (1/(2*pi))*trapz(t, integrand);
    end
    density = real(density);
end
function phi = phi_function(t, df)
    nu = df/2;
    z = sqrt(df)*abs(t);
    phi = zeros(size(t));
    zero_idx = (t == 0);
    phi(zero_idx) = 1;
    pos_idx = ~zero_idx;
    z_pos = z(pos_idx);
    K_nu = besselk(nu, z_pos);
    num = (z_pos).^nu .* K_nu;
    denom = gamma(nu)*2^(nu-1);
    phi(pos_idx) = num./denom;
    phi = phi .* exp(1i*t*0);
end
```

MATLAB Code for Task 3 and 4: Expected Shortfall Calculation

Listing 17: MATLAB Code for Expected Shortfall

```
nu = 5;
ES = compute_ES_student_t(nu);
fprintf('ES with nu = %d: %.4f\n', nu, ES);
function ES = compute_ES_student_t(nu, location, scale, ESlevel)
    if ~exist('location','var') || isempty(location)
        location = 0;
    end
    if ~exist('scale','var') || isempty(scale)
        scale = 1;
    end
    if ~exist('ESlevel','var') || isempty(ESlevel)
        ESlevel = 0.05;
    end
    alpha = ESlevel;
    VaR_standard = tinv(alpha, nu);
    pdf_VaR_standard = tpdf(VaR_standard, nu);
    ES_standard = -((nu+VaR_standard^2)/((nu-1)*alpha))*pdf_VaR_standard;
    ES = location+scale*ES_standard;
end
```

Listing 18: MATLAB Code for Plot of VaR and Expected Shortfall

```
nu = input('Enter df (nu): ');
conf_level = input('Enter conf level (conf_level, e.g., 0.95): ');
VaR = tinv(1-conf_level, nu);
pdf_VaR = tpdf(VaR, nu);
ES = - ((nu+VaR^2) / ((nu-1)*(1-conf_level)))*pdf_VaR;
fprintf('Expected Shortfall at conf. level %.2f: %.4f\n', conf_level, ES);
x_min = min(-5, ES-1);
x_max = max(5, -VaR+1);
x = linspace(x_min, x_max, 1000);
pdf_values = tpdf(x, nu);
figure('Color', 'w', 'Position', [100, 100, 800, 600]);
plot(x, pdf_values, 'b', 'LineWidth', 2);
hold on;
x_fill = linspace(x_min, VaR,100);
y_fill = tpdf(x_fill, nu);
fill([x_fill, VaR], [y_fill, 0], 'r', 'FaceAlpha', 0.5, 'EdgeColor', 'none');
plot([VaR, VaR], [0, tpdf(VaR, nu)], 'k--', 'LineWidth', 2);
plot([ES, ES], [0, tpdf(ES, nu)], 'g--', 'LineWidth', 2);
text(VaR, tpdf(VaR, nu)*1.05, sprintf(' VaR = %.2f', VaR), 'FontSize', 12, 'Color'
    , 'k', 'HorizontalAlignment', 'left', 'VerticalAlignment', 'bottom');
text(ES, tpdf(ES, nu)*1.05, sprintf(' ES = %.2f', ES), 'FontSize', 12, 'Color', 'g
    ', 'HorizontalAlignment', 'right', 'VerticalAlignment', 'bottom');
xlabel('Loss', 'FontSize', 14);
ylabel('Probability Density', 'FontSize', 14);
legend('t-Distribution PDF', 'VaR Region', 'VaR Threshold', 'ES Threshold', '
    Location', 'Best');
grid on;
box on;
xlim([x_min, x_max]);
ylim([0, max(pdf_values)*1.1]);
```

MATLAB Code for Task 5: Bootstrap Estimation

Listing 19: MATLAB Code for t-Density Plot

```
function simulate_ES_bootstrap_performance(sim, df, n, B)
    if nargin < 4 || isempty(B)
        B = 500;
    end
    if nargin < 3 || isempty(n)
        n = 250;
    end
    if nargin < 2 || isempty(df)
        df = 4;
    end
    if nargin < 1 || isempty(sim)
        sim = 1000;
    end
    coverage_parametric = zeros(sim, 1);
    coverage_nonparametric = zeros(sim, 1);
    length_parametric = zeros(sim, 1);
    length_nonparametric = zeros(sim, 1);
    ES_true = compute_ES_student_t(df, 0, 1, 0.05);
    for s = 1:sim
        data = trnd(df, n, 1);
        CI_parametric = compute_ES_parametric_bootstrap_single(data, B);
        coverage_parametric(s) = (CI_parametric(1) < ES_true) && (CI_parametric(2)
            > ES_true);
        length_parametric(s) = CI_parametric(2)-CI_parametric(1);
        CI_nonparametric = compute_ES_nonparametric_bootstrap_single(data, B, n);
        coverage_nonparametric(s) = (CI_nonparametric(1) < ES_true) && (
            CI_nonparametric(2) > ES_true);
        length_nonparametric(s) = CI_nonparametric(2)-CI_nonparametric(1);
    end
    empirical_coverage_parametric = mean(coverage_parametric)*100;
    empirical_coverage_nonparametric = mean(coverage_nonparametric)*100;
    avg_length_parametric = mean(length_parametric);
    avg_length_nonparametric = mean(length_nonparametric);
    fprintf('Emp. Covergage (Param.): %.2f%%\n', empirical_coverage_parametric);
    fprintf('Emp. Coverage (Nonparam.): %.2f%%\n',
        empirical_coverage_nonparametric);
    fprintf('Avg CI (Param): %.4f\n', avg_length_parametric);
    fprintf('Avg CI (Non-Nonparam): %.4f\n', avg_length_nonparametric);
end
function CI_parametric = compute_ES_parametric_bootstrap_single(data, B)
    ESlevel = 0.05;
    mle_params = mle(data, 'distribution', 'tLocationScale');
    nu_mle = mle_params(3);
    location_mle = mle_params(1);
    scale_mle = mle_params(2);
    ES_bootstrap_p = zeros(B, 1);
    for b = 1:B
        bootstrap_sample_p = location_mle+scale_mle*trnd(nu_mle, length(data), 1);
        VaR_bootstrap_p = quantile(bootstrap_sample_p, ESlevel);
        ES_bootstrap_p(b) = mean(bootstrap_sample_p(bootstrap_sample_p <=
            VaR_bootstrap_p));
    end
    lower_p = prctile(ES_bootstrap_p, 5);
    upper_p = prctile(ES_bootstrap_p, 95);
    CI_parametric = [lower_p, upper_p];
```

```
end
function CI_nonparametric = compute_ES_nonparametric_bootstrap_single(data, B, n)
    ES_bootstrap_np = zeros(B, 1);
    ESlevel = 0.05;
    for b = 1:B
        ind = unidrnd(n, [n, 1]);
        bootsamp = data(ind);
        VaR_bootstrap_np = quantile(bootsamp, ESlevel);
        ES_bootstrap_np(b) = mean(bootsamp(bootsamp <= VaR_bootstrap_np));
    end
    lower_np = prctile(ES_bootstrap_np, 5);
    upper_np = prctile(ES_bootstrap_np, 95);
    CI_nonparametric = [lower_np, upper_np];
end
function ES = compute_ES_student_t(nu, location, scale, ESlevel)
    if ~exist('location', 'var') || isempty(location)
        location = 0;
    end
    if ~exist('scale', 'var') || isempty(scale)
        scale = 1;
    end
    if ~exist('ESlevel', 'var') || isempty(ESlevel)
        ESlevel = 0.05;
    end
    alpha = ESlevel;
    VaR_standard = tinv(alpha, nu);
    pdf_VaR_standard = tpdf(VaR_standard, nu);
    ES_standard = -((nu+VaR_standard^2)/((nu-1)*alpha)) *pdf_VaR_standard;
    ES = location+scale*ES_standard;
end
```

MATLAB Code for Task 6: Coverage Probability Simulation

Listing 20: Full MATLAB Code for Exercise 6

```
function simulate_ES_bootstrap_performance(sim, df, n, B)
    if nargin < 4 || isempty(B)
        B = 500;
    end
    if nargin < 3 || isempty(n)
        n = 250;
    end
    if nargin < 2 || isempty(df)
        df = 4;
    end
    if nargin < 1 || isempty(sim)
        sim = 1000;
    end
    coverage_parametric = zeros(sim, 1);
    coverage_nonparametric = zeros(sim, 1);
    length_parametric = zeros(sim, 1);
    length_nonparametric = zeros(sim, 1);
    ES_true = compute_ES_student_t(df, 0, 1, 0.05);
    for s = 1:sim
        data = trnd(df, n, 1);
        CI_parametric = compute_ES_parametric_bootstrap_single(data, B);
        coverage_parametric(s) = (CI_parametric(1) < ES_true) && (CI_parametric(2)
            > ES_true);
        length_parametric(s) = CI_parametric(2)-CI_parametric(1);
        CI_nonparametric = compute_ES_nonparametric_bootstrap_single(data, B, n);
        coverage_nonparametric(s) = (CI_nonparametric(1) < ES_true) && (
            CI_nonparametric(2) > ES_true);
        length_nonparametric(s) = CI_nonparametric(2)-CI_nonparametric(1);
    end
    empirical_coverage_parametric = mean(coverage_parametric) * 100;
    empirical_coverage_nonparametric = mean(coverage_nonparametric) * 100;
    avg_length_parametric = mean(length_parametric);
    avg_length_nonparametric = mean(length_nonparametric);
    fprintf('Empirical Coverage (Parametric): %.2f%%\n',
        empirical_coverage_parametric);
    fprintf('Empirical Coverage (Non-Parametric): %.2f%%\n',
        empirical_coverage_nonparametric);
    fprintf('Average CI Length (Parametric): %.4f\n', avg_length_parametric);
    fprintf('Average CI Length (Non-Parametric): %.4f\n', avg_length_nonparametric
        );
end
function CI_parametric = compute_ES_parametric_bootstrap_single(data, B)
    ESlevel = 0.05;
    mle_params = mle(data, 'distribution', 'tLocationScale');
    nu_mle = mle_params(3);
    location_mle = mle_params(1);
    scale_mle = mle_params(2);
    ES_bootstrap_p = zeros(B, 1);
    for b = 1:B
        bootstrap_sample_p = location_mle+scale_mle*trnd(nu_mle, length(data), 1);
        VaR_bootstrap_p = quantile(bootstrap_sample_p, ESlevel);
        ES_bootstrap_p(b) = mean(bootstrap_sample_p(bootstrap_sample_p <=
            VaR_bootstrap_p));
    end
    lower_p = prctile(ES_bootstrap_p, 5);
```

```
    upper_p = prctile(ES_bootstrap_p, 95);
    CI_parametric = [lower_p, upper_p];
end

function CI_nonparametric = compute_ES_nonparametric_bootstrap_single(data, B, n)
    ES_bootstrap_np = zeros(B, 1);
    ESlevel = 0.05;
    for b = 1:B
        ind = unidrnd(n, [n, 1]);
        bootsamp = data(ind);
        VaR_bootstrap_np = quantile(bootsamp, ESlevel);
        ES_bootstrap_np(b) = mean(bootsamp(bootsamp <= VaR_bootstrap_np));
    end
    lower_np = prctile(ES_bootstrap_np, 5);
    upper_np = prctile(ES_bootstrap_np, 95);
    CI_nonparametric = [lower_np, upper_np];
end

function ES = compute_ES_student_t(nu, location, scale, ESlevel)
    if ~exist('location', 'var') || isempty(location)
        location = 0;
    end
    if ~exist('scale', 'var') || isempty(scale)
        scale = 1;
    end
    if ~exist('ESlevel', 'var') || isempty(ESlevel)
        ESlevel = 0.05;
    end
    alpha = ESlevel;
    VaR_standard = tinv(alpha, nu);
    pdf_VaR_standard = tpdf(VaR_standard, nu);
    ES_standard = -((nu+VaR_standard^2)/((nu-1)*alpha))*pdf_VaR_standard;
    ES = location+scale*ES_standard;
end
```

Listing 21: MATLAB Code for my m out of n interpretation

```
function CI = compute_ES_nonparametric_bootstrap_single(data, B, m)
    ESlevel = 0.05;
    ES_bootstrap_np = zeros(B, 1);
    for b = 1:B
        bootsamp = datasample(data, m, 'Replace', true);
        VaR_bootstrap_np = quantile(bootsamp, ESlevel);
        ES_bootstrap_np(b) = mean(bootsamp(bootsamp <= VaR_bootstrap_np));
    end
    lower_np = quantile(ES_bootstrap_np, 0.05);
    upper_np = quantile(ES_bootstrap_np, 0.95);
    CI = [lower_np, upper_np];
end
```